Logbook — Anomalous transport in soaring flights

Matteo Di Sante

Working log

Working notebook. The timeline is regenerated from the git history; the narrative notes are written by hand.

1 Timeline (auto-generated from git)

Date	Event (commit subject)
2026-07-01	Rename soaring-ffvl -> soaring-para; add hang-glider data and update all docs
2026-07-01	Fix delta config: season start 2001, data_root /Volumes/SSD_DISANTE/hang_gliders/delta_cfd_igc
2026-07-01	Add soaring-delta CLI for hang-glider CFD data (delta.ffvl.fr)
2026-07-01	Add analysis sub-package and preprocessing diagnostics; revise thesis chapters 1-4
2026-07-01	Expand pre-processing: explicit cleaning and filtering sections
2026-07-01	Clarify stationarity/compatibility check paragraph
2026-07-01	Add preliminary characterization section before main observables
2026-07-01	Add isotropy test protocol to the propagator item
2026-07-01	Rewrite propagator item: formal derivation of marginal scaling
2026-07-01	Reorganize TA-MSD item for clarity and typographic structure
2026-07-01	Clarify ergodicity breaking and add CTRW appendix
2026-06-28	Revise chapter 4 observables and the IGC description
2026-06-28	Warn when thesis and logbook next-steps drift apart
2026-06-28	Align logbook next steps with the thesis planning chapter
2026-06-28	Publish the logbook alongside the thesis
2026-06-28	Add living thesis document and build automation
2026-06-26	Deploy docs via GitHub Pages Actions artifact instead of gh-pages branch
2026-06-26	Fail fast with a clear message when data_root is unset
2026-06-26	Add 'verify' subcommand and remove machine-specific data_root path
2026-06-26	Link published docs in README; drop Development section
2026-06-25	Add GitHub Pages deployment via gh-pages branch
2026-06-25	Translate all documentation, comments, and strings to English
2026-06-25	Initialize thesis repo: FFVL CFD .igc data acquisition

2 Narrative notes

2026-06 — Paraglider data acquisition complete. The full FFVL paraglider CFD archive (`parapente.ffvl.fr`, seasons 1999–2000 to 2025–2026) has been scraped: about 203,000 flight records, of which ∼186,000 carry a GPS track. All 186,052 available tracks were downloaded. *What worked:* impersonating a Chrome TLS fingerprint (`curl_cffi`) reliably passes the FFVL

Cloudflare challenge; the resumable, atomic, validated download survived many interruptions without corruption. *What did not / gotchas:* 173 tracks could not be obtained — 172 because the server returns something that is not a valid IGC (broken or placeholder links on FFVL's side, so a retry does not help), and 1 due to a transient Cloudflare challenge (worth a later retry). On macOS/exFAT, the OS scatters `._*` sidecar files next to every written file; a `clean` step removes them.

2026-06 — Integrity verified. A full on-disk re-scan (`verify`) confirmed that all 186,052 downloaded files are structurally valid IGC: no truncated files, no leftover `.part`, no read errors. The verification logic was promoted from a throwaway script to a first-class `soaring-para verify` subcommand (reusing the download-time validator), with tests.

2026-06 — Robustness of the data path. The external disk remounted under a different name (`HDD_DISANTE` → `SSD_DISANTE`), which would have silently broken a hard-coded path. Fix: removed every machine-specific path from the repo (placeholder + `SOARING_PARA_DATA_ROOT` environment variable) and added a start-up guard that aborts with a clear, actionable message when `data_root` is unset/unmounted — and verified that `/Volumes` is not user-writable, so a wrong path can never cause a silent download to the wrong place.

2026-07 — Hang-glider data acquired; naming corrected. The hang-glider CFD (`delta.ffvl.fr`, seasons 2001–2002 to 2025–2026) was added as a second source. The acquisition code (`soaring.acquisition.ffvl`) is shared; a second CLI entry point `soaring-delta` (config `delta_download.yaml`, env var `SOARING_DELTA_DATA_ROOT`) drives it. Data: ∼9,300 flights, 6,716 tracks downloaded, 30 failures (broken server-side links, same pattern as paragliders). Stored on the SSD at `/Volumes/SSD_DISANTE/hang_gliders/delta_cfd_igc/`. At the same time the paraglider CLI was renamed from the generic `soaring-ffvl` to `soaring-para` (and `SOARING_FFVL_DATA_ROOT` → `SOARING_PARA_DATA_ROOT`), and the config was renamed from `ffvl_download.yaml` to `para_download.yaml`, to avoid ambiguity now that both sources are present.

2026-07 — Hang-glider catalog built; datasets reorganized symmetrically. Ran the full pipeline on the hang-glider data (`soaring-delta clean`, `verify`, `build-catalog`, `status`), which so far had only been downloaded: 9,259 flights, 6,746 with a track, 6,716 downloaded and all verified (0 integrity failures), 30 unrecoverable server-side links. The local `data/` folder was reorganized to mirror the SSD – one directory per discipline (`paragliders/`, `hang_gliders/`), each with a versioned `seasons_index.csv` and a git-ignored `catalog.csv`. `generate_stats.py` now reads both indices and emits per-discipline macros (`\StatPara*`, `\StatHang*`) and two per-season tables, and the first three thesis chapters were made symmetric across the two disciplines.

2026-07 — Altitude channel decision: barometric. Decided to take the vertical dynamics from the *barometric* altitude rather than the GNSS altitude, diverging from the reference pipeline (Jérémie's code derives z and v_z from `AltGNSS`, reading but not using `AltPre`). Rationale: the pressure altitude has sub-metre relative precision and low high-frequency noise, its errors being slow (offset, drift) and killed by differencing, whereas the GNSS vertical carries a high-frequency noise floor that derivatives amplify. Confirmed empirically with a new noise diagnostic – an IGC `B`-record parser (`soaring.analysis.igc`) and an altitude-noise analysis (`soaring.analysis.altitude_noise`) whose Welch power spectra show the GNSS floor two to three orders of magnitude above the barometric one near Nyquist (with a thesis appendix on the method). *Consequences:* the `A/V` validity flag is not a separate rule but is subsumed by the missing-altitude check on the adopted channel (a `V` fix keeps position + baro on a baro flight,

but is dropped on a GNSS-fallback flight); one altitude channel per trajectory, no baro/GNSS splicing; flights with no pressure sensor fall back to unfiltered GNSS, tagged by a per-flight `alt_source` column (fractions — a substantial minority, larger for paragliders — to confirm on a larger sample once the SSD is remounted); the trimming threshold is on the horizontal speed only ($v_{xy} > 5\,\mathrm{m/s}$, sustained); and the cleaning/trimming run on the raw geographic coordinates (great-circle speeds), with the ENU conversion done *last*, origin at take-off. Also redesigned the noise figure (dropped the redundant HF-index panel; panel (a) now shows the GNSS−baro difference).

2026-07 — Baro-absence fraction: sized sampling, not a full sweep. A full-population parse of the paraglider archive (186,052 files) for the baro-channel-absence figure took an unacceptable ∼45 min serially; a first fix (parallelising across 8 processes) cut this to ∼15 min, still too slow to be run routinely. Replaced it with a properly justified *simple random sample*: the standard proportion sample-size formula ($n = z^2 p(1-p)/e^2$, conservative $p = 0.5$) gives $n = 2401$ for a target 95%-confidence margin of error of ±2 percentage points – `required_sample_size` in `soaring.analysis.altitude_noise`, with `proportion_ci` reporting the resulting point estimate and interval. The population is treated as effectively infinite (no finite-population correction): at $n = 2401$ out of 186,052 paraglider flights (∼1.3% sampled), the correction would move n by only about 30, not worth the extra parameter. The hang-glider archive (6716 files) stays a full census: cheap enough (under two minutes) that there is nothing to gain from sampling it. Total runtime: ∼50 s. Final figures: paragliders $(28.4 \pm 1.8)\,\%$ (95% CI, sampled), hang gliders 17.5 % (exact, census) – consistent with the earlier, less rigorously sized estimates. *Also caught mid-flight:* the IGC parser accepted syntactically well-formed but semantically invalid `B` records (e.g. minute ≥ 60, hour ≥ 24, out-of-range latitude/longitude) without rejecting them; added explicit format-validity checks (9 new tests) and corrected the thesis wording that had conflated this *implemented* format-validity layer with the fix-level *dynamics-plausibility* layer of Table 4.1, which is designed (thresholds exist as a dataclass) but not yet implemented as code – an important distinction the thesis now states explicitly rather than glossing over.

2026-06 — Documentation. Project docs are built with MkDocs and published on GitHub Pages. Initially deployed with `mkdocs gh-deploy`, which accumulated one built-site commit per deploy on a `gh-pages` branch; migrated to the official GitHub Pages *artifact* workflow (build → upload artifact → deploy), removing the branch and keeping the git history clean.

Operational note. In this repo `uv sync` occasionally fails to (re)install the editable package; `uv sync -reinstall-package soaring` fixes it.

3 Next steps

These mirror the planning chapter of the state document (`thesis/sections/04-next-steps`) and are kept in sync with it. The near-term plan is to analyse the data *in parallel* with studying the segmentation, since many observables can be built before any phase splitting.

- **Data sources.** FFVL paraglider and hang-glider CFD data are collected, cataloged and verified. Extend to further sources – notably WeGlide for sailplanes (a higher-quota request is pending) and possibly XContest – merging into one catalog and stratifying by source. Refresh the per-discipline `seasons_index.csv` via `build-catalog` so the document statistics stay current.
- **Pre-processing.** The IGC `B`-record parser is in place (`soaring.analysis.igc`); next, turn parsed fixes into clean, resampled, quality-filtered trajectories in a local metric frame,

using the barometric altitude for the vertical channel (fix-level cleaning, v_{xy}-based trimming, flight-level filtering, resampling).

- **Whole-trajectory observables (no segmentation).** Immediate next step: characterise the data with observables that need no phases — MSD and its scaling exponent, the displacement distribution and its tails, the velocity autocorrelation and turning-angle distribution, the speed/vertical-velocity distributions, and geometric measures (tortuosity, fractal dimension, radius of gyration). Goal: a first physical/statistical picture and some preliminary findings.
- **Segmentation.** Study Jérémie's internship report and code, then re-examine the transition/search/climb HMM (binary features: sign of vertical speed, curvature-angle persistence, straightness) — decide whether to keep that approach or add complexity for a better segmentation on our larger, more heterogeneous data.
- **Phase-resolved observables.** After segmentation, extract step lengths, waiting times, search durations, per-phase kinematics and the angular diffusion between successive glides.
- **Reproduce the reference.** Check that the enlarged dataset reproduces Vilpellet et al.: per-phase conditional MSD (their Fig. 3) and per-phase descriptive statistics (their Fig. 4), plus the global $H \approx 0.88$ — validation + universality test.
- **Transport analysis.** Estimate the MSD scaling exponent, fit the tails of the waiting-time and step-length distributions, and perform CTRW-vs-Lévy-walk model selection.
- **Minimal model.** A minimal stochastic model for $H \approx 0.88$: resolve transition segments, merge search+climb into one effective state, with angular diffusion between successive transition directions (preliminary idea, to be refined/changed).